

SCRAPL: Scattering Transform with Random Paths for Machine Learning

Christopher Mitcheltree^b Vincent Lostanlen[#] Emmanouil Benetos^b Mathieu Lagrange[#]

^b Centre for Digital Music, Queen Mary University of London, UK

[#] Nantes Université, École Centrale Nantes, CNRS, LS2N, UMR 6004, France

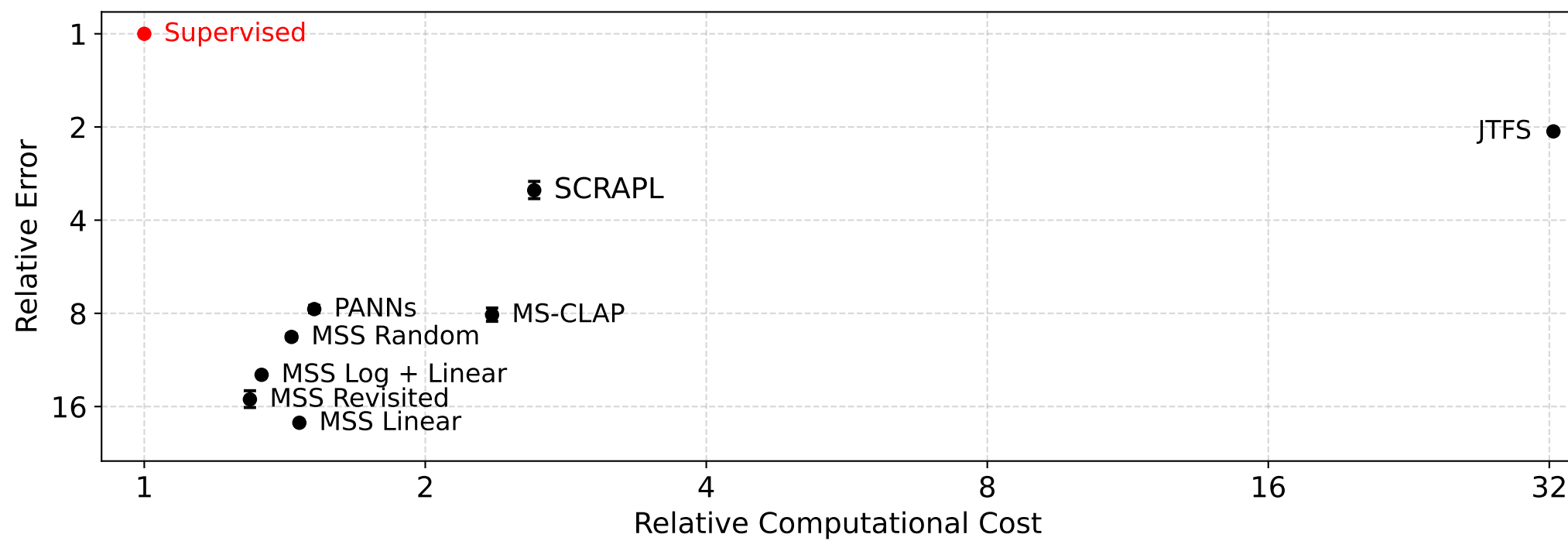


Figure 1. Mean synthesizer parameter error versus computational cost of unsupervised sound matching models for the granular synthesis task. Both axes are rescaled by the performance of a supervised model with the same number of parameters.

Overview

- **TLDR:** the SCRAPL algorithm enables efficient evaluation of multivariable wavelet scattering transforms as differentiable loss functions for neural network training.
`pip install scrapl-loss` [christhetree.github.io/scrapl](https://github.com/christhetree/scrapl)
- **Motivation:** The Euclidean distance between wavelet scattering transform coefficients (known as *paths*) provides informative gradients for perceptual quality assessment in deep inverse problems for computer vision, speech, and audio.
- **Problem:** These transforms are computationally expensive when employed as differentiable loss functions, which significantly limits their use in neural networks.
- **Approach:** We propose SCRAPL: a stochastic optimization scheme for the efficient evaluation of multivariable scattering transforms. It stabilizes inexpensive single-path gradients using path-wise adaptive moment estimation ($\mathcal{P}$ -Adam), path-wise stochastic average gradient with acceleration ($\mathcal{P}$ -SAGA), and an architecture-informed, parallelizable θ -importance sampling initialization heuristic.
- **Experiments:** We apply SCRAPL to the joint time–frequency scattering transform (JTFS) and unsupervised sound matching with differentiable digital signal processing (DDSP) of a granular synthesizer and the Roland TR-808 drum machine.
- **Findings:** SCRAPL accomplishes a favorable tradeoff between goodness of fit and computational efficiency: it is within a factor two of full-tree scattering (JTFS) for accuracy, while remaining within a factor two of multiscale spectral loss (MSS) for runtime. Furthermore, the θ -importance sampling heuristic improves convergence.

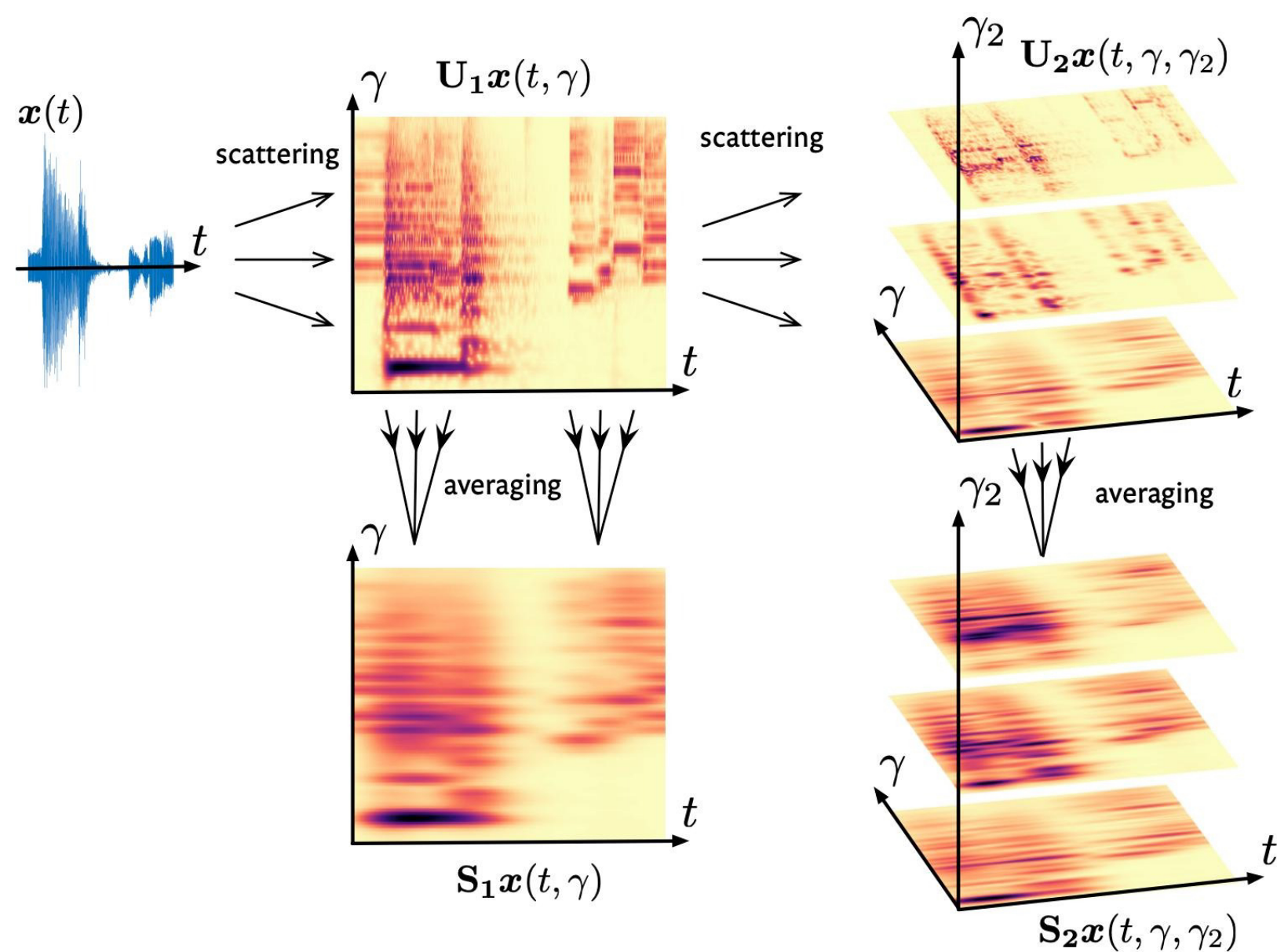


Figure 2. Visualization of a wavelet scattering transform applied to an input signal $x(t)$.

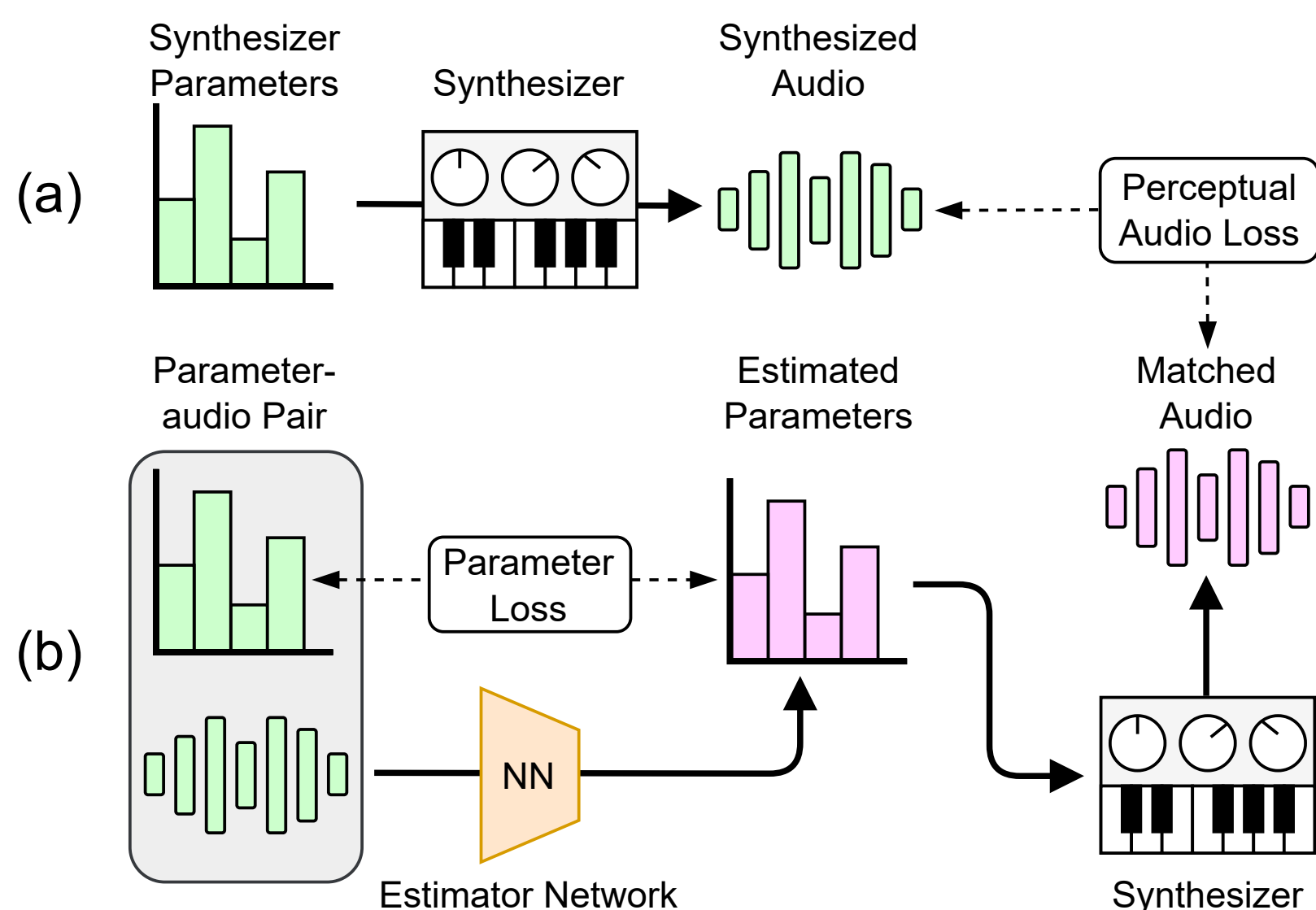


Figure 3. Unsupervised (a) and supervised (b) sound matching using DDSP.

Algorithm 1 “Scattering transform with Random Paths for machine Learning” (SCRAPL). The pseudo-code below describes SCRAPL with a batch size equal to one, without loss of generality.

```

Require:  $\Phi = (\phi_p)_0^{P-1}$ : Scattering transform (ST) with  $P$  paths
Require:  $\pi$ : Categorical distribution over the path set  $\mathcal{P} = \{0, \dots, P-1\}$ 
Require:  $F$ : Autoencoder with trainable parameters  $w$ 
Require:  $w$ : Neural network weights at initialization
Require:  $\beta_1, \beta_2, \epsilon$ : Adam hyperparameters
Require:  $(\alpha_k)_1^K$ : Learning rate schedule
 $\Gamma \leftarrow \emptyset$ 
for  $p$  in  $\{0, \dots, P-1\}$  do
   $\tau_p \leftarrow 0$ 
   $m_p \leftarrow 0$ 
   $v_p \leftarrow 0$ 
   $\hat{g}_p \leftarrow 0$ 
end for
for  $k$  in  $\{1, \dots, K\}$  do
   $n \leftarrow$  draw an integer uniformly at random in  $\{0, \dots, N-1\}$ 
   $p \leftarrow$  draw an integer at random in  $\{0, \dots, P-1\}$  according to  $\pi$  {Stochastic approx.}
   $\mathcal{L}(w) \leftarrow P \|\phi_p(x_n) - (\phi_p \circ F_w)(x_n)\|_2^2$ 
   $g \leftarrow \nabla \mathcal{L}(w)$ 

   $m_p \leftarrow \beta_1^{(k-\tau_p)/P} m_p + (1 - \beta_1^{(k-\tau_p)/P}) g$ 
   $v_p \leftarrow \beta_2^{(k-\tau_p)/P} v_p + (1 - \beta_2^{(k-\tau_p)/P}) (g \odot g)$ 
   $\hat{m} \leftarrow m_p / (1 - \beta_1^{k/P})$  {P-Adam}
   $\hat{v} \leftarrow v_p / (1 - \beta_2^{k/P})$ 
   $g_{\text{current}} \leftarrow \hat{m} / \sqrt{\epsilon + \hat{v}}$ 
   $\tau_p \leftarrow k$ 

   $g_{\text{avg}} \leftarrow \frac{1}{\max(1, \text{card}(\Gamma))} \sum_{\gamma \in \Gamma} \hat{g}_\gamma$ 
   $g_{\text{SAGA}} \leftarrow g_{\text{current}} - \hat{g}_p + g_{\text{avg}}$ 
   $w \leftarrow w - \alpha_k g_{\text{SAGA}}$  {P-SAGA}
   $\hat{g}_p \leftarrow g_{\text{current}}$ 
   $\Gamma \leftarrow \Gamma \cup \{p\}$ 
end for
return  $w$ 

```

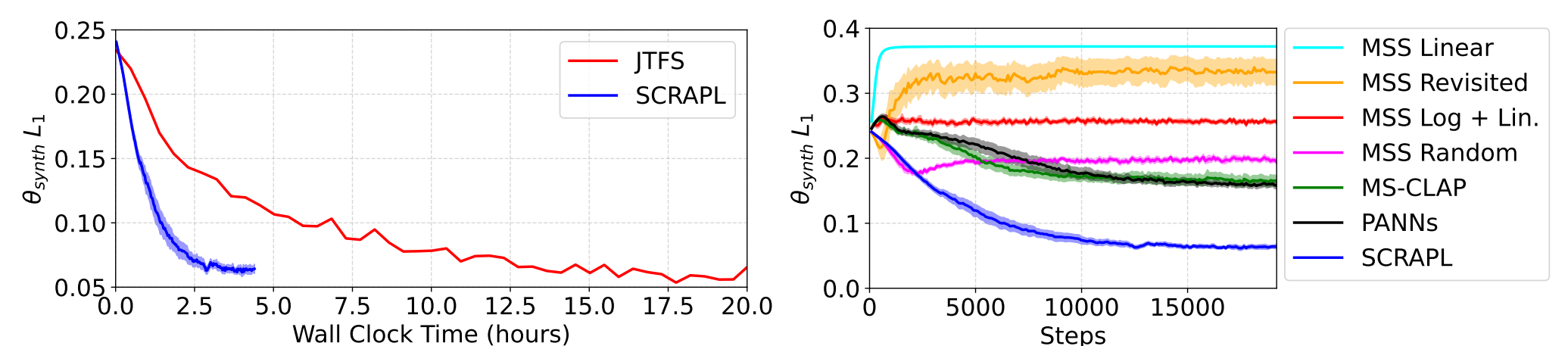


Figure 4. Left: JTFS vs. SCRAPL wall-clock training times on a single RTX A5000 GPU. Right: Validation convergence graphs for the granular synth sound matching task.

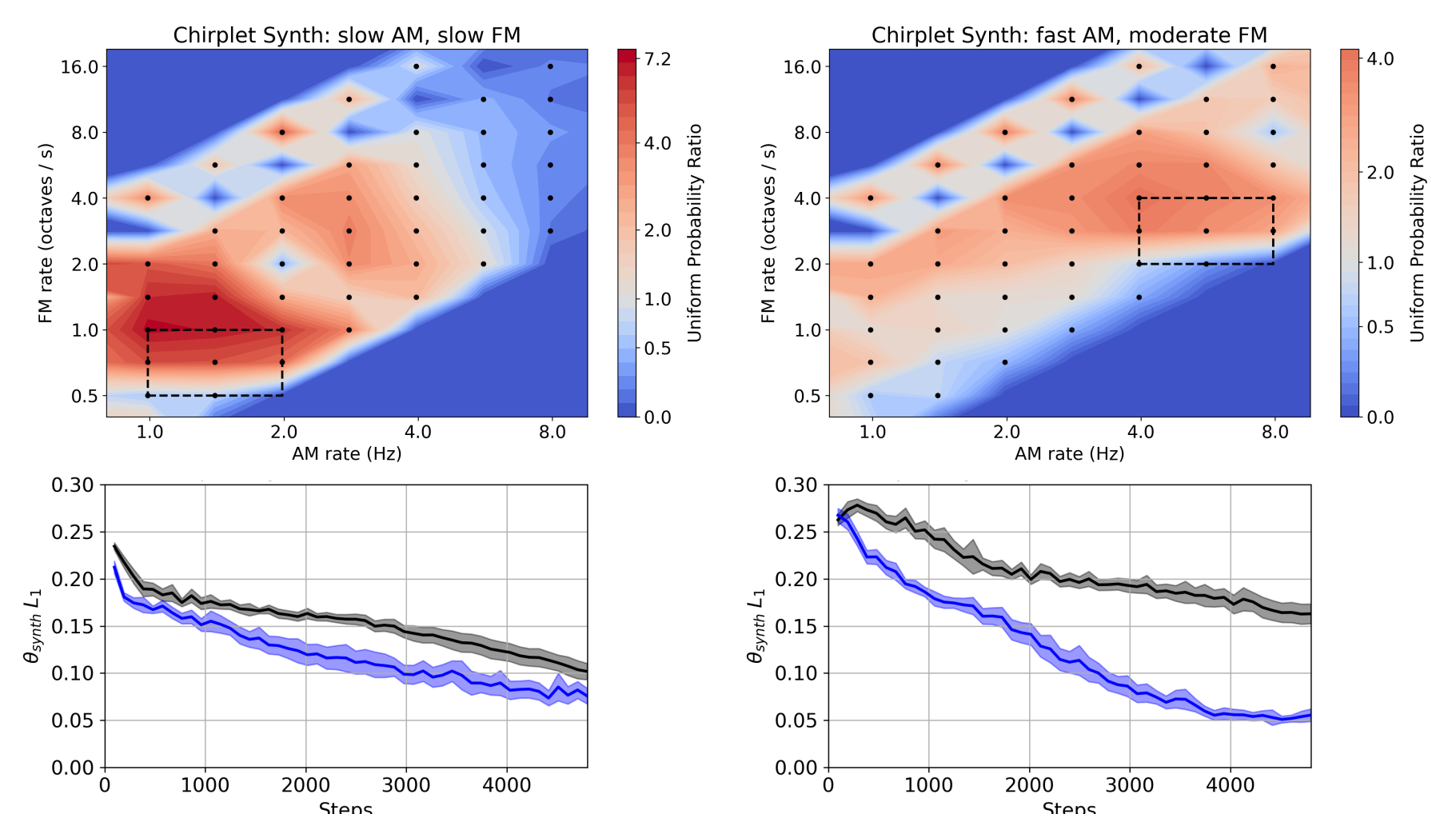


Figure 5. Top: SCRAPL path θ -importance sampling probabilities for two different AM / FM chirplet synths calculated from 1 batch of 32 log-uniformly randomly sampled θ_{synth} values. Black dots are individual path (wavelet) AM / FM center frequency locations and each dashed rectangle is the θ_{synth} range for each synth configuration. A uniform probability ratio of 1.0 means a path is sampled with probability $1/P$. Bottom: θ -importance sampling (blue) vs. uniform sampling (black) $\theta_{\text{synth}} L_1$ validation values during training for two different AM / FM chirplet synths.

Table 1. Ablation table for SCRAPL with test results and validation $\theta_{\text{synth}} L_1$ total variation and convergence steps for unsupervised granular synth sound matching.

Method	$\mathcal{P}$ -Adam	$\mathcal{P}$ -SAGA	θ -IS	Test		Validation	
				$\theta_{\text{synth}} L_1$	% $\downarrow$	Total Var. $\downarrow$	Conv. Steps $\downarrow$
SCRAPL				99.7 $\pm$ 8.2		5.30 $\pm$ 0.25	10 906 $\pm$ 1170
	✓			87.4 $\pm$ 15		6.98 $\pm$ 0.25	8006 $\pm$ 697
	✓	✓		73.8 $\pm$ 13		3.46 $\pm$ 0.15	7296 $\pm$ 683
	✓	✓	✓	65.7 $\pm$ 4.2		3.27 $\pm$ 0.12	6014 $\pm$ 642
JTFS				42.4		5.66	1442
P-loss				20.5 $\pm$ 0.20		1.83 $\pm$ 0.025	672 $\pm$ 23